

Day 47: Classic CNN Architectures I -- Recreating LeNet-5 by Hand

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 7, Day 6

Day 46 closed out this series' recent detour into learning-rate schedules with a plain admission: neither schedule fixed Day 44's stubborn wide network, because a schedule fixes rate-driven oscillation, not a network that's merely slow to finish. Today moves past hyperparameter tuning into architecture design proper -- recreating a real, named historical network, LeNet-5, by hand, and honestly checking whether following its exact original choices (tanh, average pooling, three fully-connected layers) beats this series' own much simpler baseline on the same small dataset. It doesn't, and the reason why is the actual lesson.

0. Where Today Fits

Start here (no prior knowledge assumed): Every network built since Day 43 -- TwoBlockConvNetRGB and its wide variant -- used the same generic, 'modern-ish' recipe: ReLU activations, max pooling, one hidden dense layer. Nobody designed that recipe on purpose to match a famous paper; it was just a reasonable, working default. Today is different: it recreates an actual named, historical architecture -- LeNet-5, published by Yann LeCun and colleagues in 1998 -- following its SPECIFIC original choices on purpose, then honestly measures whether historical fidelity is actually an advantage on this series' own small dataset.

1. Recreating LeNet-5

Start here (no prior knowledge assumed): LeNet-5 is one of the first convolutional neural networks ever used successfully in the real world -- reading handwritten digits on bank checks in the 1990s, at a time when ReLU activations did not exist yet and 'deep learning' was not a common phrase. Its architecture is a fixed, well-documented sequence of layers, unlike the somewhat arbitrary hidden-layer sizes chosen ad hoc in Days 43-46.

Topic specification: Original LeNet-5: 32x32 grayscale input -> Conv(6 filters, 5x5) -> avg pool(2x2) -> Conv(16 filters, 5x5) -> avg pool(2x2) -> flatten -> FC(120) -> FC(84) -> FC(10 classes), tanh activations throughout, softmax-equivalent output. This lesson's LeNet5Style class keeps every one of those specific choices -- 5x5 kernels, 6 then 16 filters, tanh, average pooling, three FC layers sized 120 and 84 -- adapted only where the input itself differs: 30x30 RGB (3 channels, this series' own junction dataset) and 4 output classes instead of the original's 32x32 grayscale digits and 10 classes.

Explanation: The three fully-connected layers are the biggest structural departure from every prior day's network. Days 43-46 all used a single hidden layer between the flattened conv output and the class scores. LeNet-5 instead narrows gradually -- 256 (flattened) -> 120 -> 84 -> 4 -- which is itself a design choice with a real cost: three weight matrices to learn instead of two.

Real-world scenario: The original 1990s use case: a bank needed to automatically read the handwritten dollar amount on millions of checks, where the input was always a small, roughly-centered grayscale digit -- not unlike this series' own small, centered, synthetic junction images. That similarity in problem shape (small image, few classes, limited training data by modern standards) is exactly what makes LeNet-5 a fair architecture to test on this series' own dataset.

Mathematics:

MATH

Shape trace through both conv+pool blocks (same valid-convolution and floor-division pooling formulas used since Day 41, applied to LeNet-5's own kernel/filter sizes):

conv1: $(30 - 5 + 1) = 26 \rightarrow \text{avgpool}(2): 26 // 2 = 13$

conv2: $(13 - 5 + 1) = 9 \rightarrow \text{avgpool}(2): 9 // 2 = 4$ (9 is odd -- the trailing row/column is dropped, same convention as every non-divisible pooling since Day 41)

flatten: $16 \text{ filters} \times 4 \times 4 = 256$

*Total learnable parameters = conv1 ($6 \cdot 3 \cdot 5 \cdot 5 + 6 = 456$) + conv2 ($16 \cdot 6 \cdot 5 \cdot 5 + 16 = 2416$) + FC1 ($256 \cdot 120 + 120 = 30840$) + FC2 ($120 \cdot 84 + 84 = 10164$) + FC3 ($84 \cdot 4 + 4 = 340$) = **44,216**.*

Implementation:

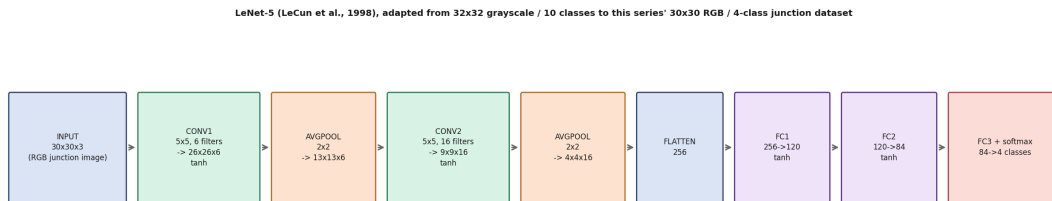
```
class LeNet5Style:
    def __init__(self, img_size=30, n_classes=4, lr=0.08, seed=42):
        rng = np.random.RandomState(seed)
        k = 5
        f1, f2 = 6, 16
        self.Wc1 = rng.randn(f1, 3, k, k) * he_scale(3 * k * k)
        self.Wc2 = rng.randn(f2, f1, k, k) * he_scale(f1 * k * k)
        ...
        self.W1 = rng.randn(self.flat_dim, 120) * he_scale(self.flat_dim)
        self.W2 = rng.randn(120, 84) * he_scale(120)
        self.W3 = rng.randn(84, n_classes) * he_scale(84)

    def forward(self, X):
        self.Ac1 = tanh(conv2d_forward_mc(X, self.Wc1, self.bc1))
        self.pooled1 = avgpool_forward(self.Ac1, 2, 2)
        self.Ac2 = tanh(conv2d_forward_mc(self.pooled1, self.Wc2, self.bc2))
        self.pooled2 = avgpool_forward(self.Ac2, 2, 2)
        self.flat = self.pooled2.reshape(X.shape[0], -1)
        self.A1 = tanh(self.flat @ self.W1 + self.b1)
        self.A2 = tanh(self.A1 @ self.W2 + self.b2)
        self.probs = softmax(self.A2 @ self.W3 + self.b3)
        return self.probs
```

Line-by-line explanation

- $k=5$, $f_1=6$, $f_2=16$ are hardcoded, not parameters -- these are LeNet-5's own historical values, not choices this lesson is free to tune.
- Every activation is $\tanh()$ -- conv layers AND both hidden FC layers. TwoBlockConvNetRGB used $\text{relu}()$ for convs and $\tanh()$ only for its single hidden layer.
- `avgpool_forward` replaces `maxpool_forward` at both pooling steps -- the one new piece of code this lesson actually writes from scratch.
- Three matrix multiplies in the FC head (W_1 , W_2 , W_3) instead of two (W_1 , W_2) -- one more layer's worth of forward and backward math.

How it works, visually:



Faithful to the original: 5x5 kernels, 6 then 16 filters, tanh throughout, average pooling, three FC layers (120 -> 84 -> classes)

LeNet-5's full shape trace, adapted to this series' 30x30 RGB, 4-class dataset.

Watch: LeNet-5 CNN Architecture Explained | The Network That Started Deep Learning — a layer-by-layer walkthrough of the exact architecture recreated in this lesson.

Why choose this? Recreating a real, published architecture teaches you to read and reproduce a paper's specification exactly -- kernel sizes, filter counts, activation choices, layer ordering -- which is a different (and equally important) skill from designing your own network from scratch, and it gives you a known, well-studied reference point to compare any future design against.

Why NOT this (tradeoffs)? LeNet-5's specific choices are 1998 choices: $\tanh$ saturates and slows learning far from zero (a known limitation ReLU was later designed to fix), and 44,216 parameters is a LOT for a 200-image training set, as today's results show directly. Faithfulness to history is not the same thing as fitness for your specific problem.

2. Average Pooling -- New From-Scratch Code

Start here (no prior knowledge assumed): Every pooling layer since Day 41 has been MAX pooling: look at a small window of the feature map, keep only the largest value, throw the rest away. Average pooling asks a different question of the same window: instead of 'which single position matters most here?', it asks 'what's the overall activation level across this whole window?' -- and answers with the mean of every value in it.

Topic specification: `avgpool_forward(A, size=2, stride=2)`: for each non-overlapping (or overlapping, depending on stride) window, `output = window.mean()`. `avgpool_backward(dOut, A_shape, size, stride)`: distributes each output position's gradient EQUALLY across every input position that contributed to it -- no index cache required, unlike `maxpool_backward`.

Explanation: Max pooling's backward pass needs to remember WHICH position won, because only that one position gets any gradient (everywhere else in the window contributed nothing to the output, so it gets zero). Average pooling has no 'winner' -- every position in the window contributed an equal $1/(\text{size} \times \text{size})$ share to the output, so an equal $1/(\text{size} \times \text{size})$ share of the output's gradient flows back to each one. This symmetry is what makes `avgpool_backward` simpler code than `maxpool_backward`, with no `argmax` or index bookkeeping at all.

Real-world scenario: Imagine averaging four neighboring pixel brightness readings versus reporting only the single brightest one. The average is more robust to one noisy or unusually bright outlier pixel; the max is more sensitive to a single strong local feature (like one bright edge in an otherwise dark patch) that would get diluted by averaging.

Mathematics:

MATH

Forward, for a $\text{size} \times \text{size}$ window W :

$$\text{output} = (1 / \text{size}^2) \cdot \sum W_{i,j}$$

Backward, for output gradient g flowing into that same window:

$$\partial L / \partial W_{i,j} = g / \text{size}^2 \text{ (the SAME value, for every } i,j \text{ in the window)}$$

Contrast with max pooling's backward rule, already used since Day 41:

$$\partial L / \partial W_{i,j} = g \text{ if } (i,j) \text{ is the argmax position, else } 0$$

Implementation:

```
def avgpool_forward(A, size=2, stride=2):
    m, C, H, W = A.shape
    out_H = (H - size) // stride + 1
    out_W = (W - size) // stride + 1
    out = np.zeros((m, C, out_H, out_W))
    for i in range(out_H):
        for j in range(out_W):
            window = A[:, :, i*stride:i*stride+size,
                        j*stride:j*stride+size]
            out[:, :, i, j] = window.mean(axis=(2, 3))
    return out

def avgpool_backward(dOut, A_shape, size=2, stride=2):
    dA = np.zeros(A_shape)
    share = 1.0 / (size * size)
    out_H, out_W = dOut.shape[2], dOut.shape[3]
    for i in range(out_H):
        for j in range(out_W):
            dA[:, :, i*stride:i*stride+size,
                j*stride:j*stride+size] += (
                dOut[:, :, i, j][:, :, None, None] * share)
    return dA
```

How it works, visually:

MAX POOLING (Days 41-46) forward: keep the largest value in each window	AVERAGE POOLING (NEW today) forward: mean of every value in each window
backward: ALL of the output gradient goes to whichever ONE input position was the max -- every other position in the window gets zero (needs an index cache to remember which one)	backward: the output gradient is split into an EQUAL share (1/window_size) for EVERY input position in the window -- no index cache needed, since every position always participates equally
example 2x2 window [[1, 4], [2, 3]]: forward output = 4 backward: gradient all goes to the position holding 4, others get 0	example 2x2 window [[1, 4], [2, 3]]: forward output = 2.5 backward: gradient split 1/4 to EACH of the 4 positions

The same 2x2 window, [[1,4],[2,3]], through both pooling rules -- forward and backward.

Watch: CNN Pooling Explained | Max Pooling, Average Pooling in Convolutional Neural Networks — covers both pooling types side by side, forward pass only -- today's lesson adds the backward-pass mechanics.

Why choose this? No index cache to maintain, a symmetric and slightly gentler gradient flow (every input position always gets SOME signal, never exactly zero), and it matches LeNet-5's own original design choice exactly.

Why NOT this (tradeoffs)? By spreading credit equally across a whole window, average pooling can dilute a single strong, spatially-localized feature -- exactly the kind of signal max pooling is specifically good at preserving. Modern architectures overwhelmingly default to max pooling (or skip pooling in favor of strided convolutions) for this reason.

3. Gradient-Checking the New Component

Start here (no prior knowledge assumed): Day 44 found a numerical gradient check can pass on an isolated function tested with its own ad hoc loss, yet still hide a real bug -- because the isolated test's loss normalization didn't match what the network's ACTUAL training loop uses. The fix, followed again today: check gradients through the exact forward()->loss->backward() pipeline LeNet5Style itself trains with, on a small input, not avgpool_forward/backward tested alone.

Explanation: Three weight tensors get checked -- Wc1 (inside the new avgpool path), W1 (the first new FC layer), and W3 (the new third FC layer) -- each at three random index positions, comparing the analytic gradient computed by backward() against a numerical estimate from nudging that one weight by $\pm 1e-4$ and re-running the full forward pass both times.

Actual output:

```
=====
1. GRADIENT CHECK -- LeNet5Style, full forward->loss->backward pipeline
=====
```

param	analytic	numeric	rel_err
Wc1(3, 1, 4, 1)	0.012681	0.012681	3.82e-09
Wc1(0, 2, 4, 4)	-0.059577	-0.059577	2.04e-09
Wc1(1, 0, 0, 0)	0.012869	0.012869	1.81e-10
W1(52, 4)	-0.007145	-0.007145	1.13e-09
W1(48, 8)	-0.003750	-0.003750	1.67e-09
W1(39, 49)	-0.007135	-0.007135	1.25e-09
W3(28, 1)	-0.003388	-0.003388	1.16e-09
W3(44, 2)	-0.015347	-0.015347	4.22e-09
W3(68, 2)	0.028652	0.028652	1.17e-09

```
max relative error across all checked entries: 4.22e-09
(PASS -- avgpool_forward/backward and the full pipeline are correct)
```

Actual output from this lesson's run.

Term refresher — relative error: $|\text{numeric} - \text{analytic}| / (|\text{numeric}| + |\text{analytic}|)$ -- used instead of plain absolute difference because it stays meaningful whether the gradient itself is large or tiny. Values below roughly $1e-4$ are considered a pass; today's worst case (4.22e-09) is far below that.

4. Training For Real: LeNet5Style vs. This Series' Baseline

Start here (no prior knowledge assumed): Both networks trained on the IDENTICAL dataset convention Days 44 and 46 used: train = seed=42, n_per_class=50 (all 200 images); test = a SEPARATE seed=999, n_per_class=20 generation (80 images). Both: lr=0.08, 400 full-batch epochs, seed=42 weight initialization.

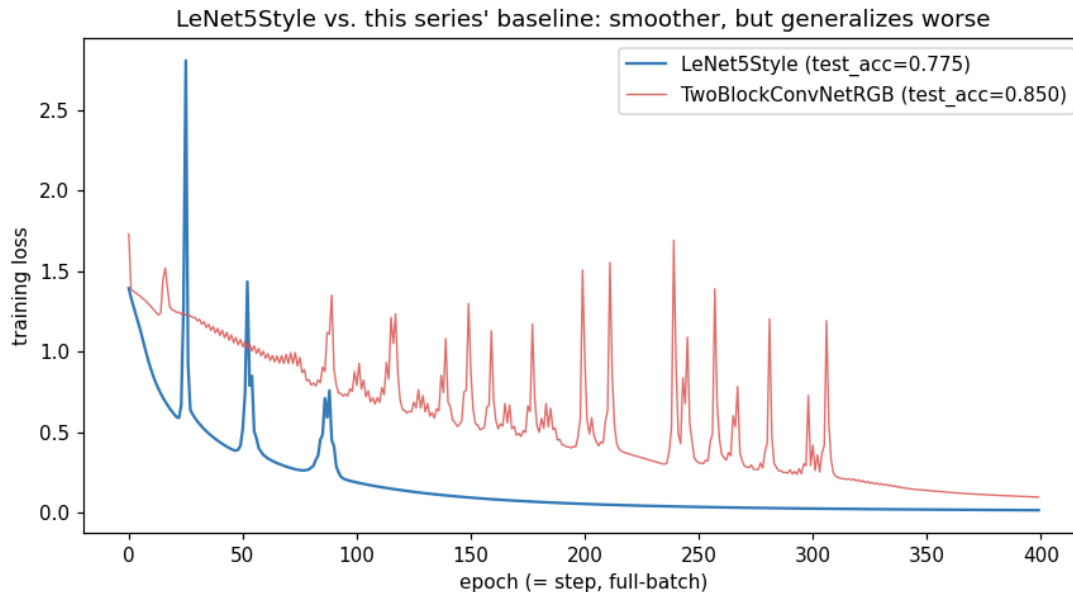
Explanation: This is the direct test of today's real question: does following LeNet-5's exact historical recipe actually help on this specific, small dataset, compared to the much simpler baseline this series has already been using since Day 43?

Actual output:

network	params	train_acc	test_acc	train-test gap
LeNet5Style	44,216	1.0000	0.7750	0.2250
TwoBlockConvNetRGB	2,024	1.0000	0.8500	0.1500

Actual output from this lesson's run.

How it works, visually:



Full 400-epoch training loss for both networks on identical data.

Explanation: LeNet5Style has 21.8x more parameters (44,216 vs. 2,024) than this series' own baseline, and both networks reach train_acc=1.0000 -- perfect memorization of all 200 training images. But LeNet5Style generalizes WORSE: test_acc=0.7750 vs. the baseline's 0.8500, and its train/test gap is wider (0.2250 vs. 0.1500). This is not evidence that LeNet-5 is a bad architecture -- it was designed and validated on tens of thousands of MNIST digits, not 200 hand-generated images across 4 classes. 44,216 parameters for 200 training examples is a far worse capacity-to-data ratio than 2,024 parameters for the same 200 examples, and that mismatch shows up directly as extra overfitting.

Why choose this? A bigger, historically-validated architecture is the right choice when you actually have the data to match its capacity -- LeNet-5 on the real MNIST dataset (60,000 training images) is a completely different, well-justified matchup than LeNet-5 on 200 synthetic images.

Why NOT this (tradeoffs)? Blindly reaching for a famous architecture without checking whether its parameter count fits your actual dataset size is a common real mistake -- today's own result (44,216 params, 200 training images, a wider overfitting gap than a 2,024-param alternative) is a concrete demonstration of exactly that mismatch, not a hypothetical.

5. A Second Finding: Transient vs. Persistent Instability

Start here (no prior knowledge assumed): Looking at the loss plot above, LeNet5Style is NOT smooth from the very start -- it spikes sharply in its first ~100 epochs before settling down completely. The baseline's noise, by contrast, never really settles.

Actual output:

network	params	train_acc	test_acc	noise 1st half	noise 2nd half
LeNet5Style	44216	1.0000	0.7750	0.19680	0.000120
TwoBlockConvNetRGB	2024	1.0000	0.8500	0.14482	0.211250

LeNet5Style: peak loss 2.81 at epoch 25, then near-total smoothness by the second half of training.

TwoBlockConvNetRGB: noise is HIGHER in the second half than the first -- visible spikes recur all the way to epoch 400.

Actual output from this lesson's run (noise_std = std of consecutive loss differences).

Explanation: Days 45 and 46 already documented this exact baseline network's real late-training oscillation at fixed $\text{lr}=0.08$ on this same dataset -- today's result isn't new for the baseline. What IS new is LeNet5Style's opposite pattern: rough early, smooth late. The likely mechanism is tanh's own shape -- it saturates as weights grow during training, and its gradient shrinks toward zero at both extremes. A network built entirely from saturating activations naturally damps its own step-to-step movement over time in a way ReLU's unbounded, non-saturating activation does not. Neither network's behavior is universal -- both are properties of THEIR specific choice of activation function interacting with a fixed learning rate, not general truths about training small conv nets.

Term refresher — saturating activation: An activation function whose output flattens out (and whose gradient approaches zero) for large positive or negative inputs. tanh saturates at ± 1 ; ReLU does not saturate for positive inputs at all, which is part of why it largely replaced tanh and sigmoid in architectures designed after the 1990s.

6. What This Maps To in PyTorch

Start here (no prior knowledge assumed): Every piece of LeNet5Style has a one-to-one equivalent in a modern framework -- the historical architecture, not just this lesson's from-scratch implementation of it, is directly expressible in PyTorch.

Implementation:

```
class LeNet5(nn.Module):
    def __init__(self, n_classes=4):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 6, kernel_size=5)
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5)
        self.pool = nn.AvgPool2d(2, 2)    # matches avgpool_forward
        self.fc1 = nn.Linear(16 * 4 * 4, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, n_classes)

    def forward(self, x):
        x = self.pool(torch.tanh(self.conv1(x)))
        x = self.pool(torch.tanh(self.conv2(x)))
        x = x.flatten(1)
        x = torch.tanh(self.fc1(x))
        x = torch.tanh(self.fc2(x))
        return self.fc3(x)    # CrossEntropyLoss applies softmax
```

Explanation: nn.AvgPool2d does exactly what avgpool_forward/avgpool_backward do here -- mean over each window forward, an equal $1/(\text{size} \cdot \text{size})$ share of the output gradient to every input position backward -- just with autograd computing that backward pass automatically instead of it being written out by hand, as it was earlier today.

Interview Preparation — Technical Questions

Q: Why does `avgpool_backward` not need an index cache, unlike `maxpool_backward`?

A: Because every position in an average-pooling window contributes equally to the output (its share of the mean), so every position always gets an equal share of the gradient back -- there's no 'winner' to remember. Max pooling's backward pass needs to know WHICH position was the max, because only that one position gets any gradient at all; everywhere else in the window is multiplied by zero.

Q: `LeNet5Style` has 44,216 parameters and `TwoBlockConvNetRGB` has 2,024 -- walk through where that difference comes from.

A: Almost entirely the FC head: `LeNet5Style`'s FC1 alone (256->120) is 30,840 parameters, versus `TwoBlockConvNetRGB`'s single hidden layer (its `flat_dim`->8) which is only a few hundred. The conv layers differ too (`LeNet5Style`'s 5x5/6-then-16-filter convs vs. the baseline's 3x3/3-then-6-filter convs) but contribute far less to the total than the three-layer, 120/84-wide FC head does.

Q: Both networks reach `train_acc=1.0000`. Why is that not enough to call them equally good?

A: Perfect training accuracy only shows a network has enough capacity to memorize its training set -- it says nothing about whether it learned patterns that generalize. Today's test accuracy (0.7750 vs. 0.8500 on a separately-generated 80-image test set) is the number that actually distinguishes the two networks, and the wider gap for the bigger network is a textbook overfitting signature.

Q: Why gradient-check through the full forward->loss->backward pipeline instead of testing `avgpool_forward/backward` in isolation?

A: Day 44 found that an isolated test with its own ad hoc loss can pass even when the real training pipeline has a bug, because the isolated loss's normalization doesn't necessarily match what the actual network uses. Checking through the exact pipeline training uses removes that gap entirely.

Q: What specific choices make `LeNet5Style` 'faithful' to the original 1998 LeNet-5, versus this series' own baseline network?

A: 5x5 kernels (not 3x3), 6 then 16 filters (not 3 then 6), tanh activations throughout including both conv layers (not ReLU for convs), average pooling (not max pooling), and three fully-connected layers sized 120 and 84 (not one hidden layer).

Interview Preparation — Theoretical Questions

Q: Why might a network's training loss be noisy early on but smooth out later, purely from its choice of activation function?

A: A saturating activation like tanh has a gradient that shrinks toward zero as its input grows large in either direction. Early in training, weights (and therefore pre-activation values) can be large or poorly scaled, producing large, noisy gradient updates. As training proceeds and the network settles into a region where more of its units are saturated, those same large weight-update swings get naturally damped -- shrinking step sizes without any explicit learning-rate schedule at all.

Q: Is a historically important architecture like LeNet-5 always a reasonable default choice for a new small-data problem?

A: Not automatically. An architecture's parameter count needs to be matched to the amount of training data available, not just to how well-known or historically significant the architecture is. LeNet-5 was validated on 60,000 MNIST training images; applying it unchanged to a 200-image dataset reproduces none of the conditions that made it successful in the first place.

Q: What's the tradeoff between average pooling and max pooling in terms of what information survives to later layers?

A: Max pooling preserves the single strongest signal in each window and discards everything else, which is good for detecting a specific sharp feature (like an edge) regardless of everything around it. Average pooling preserves the overall activation level across the whole window, which is more robust to noise but can dilute a strong, spatially-narrow feature by blending it with weaker neighboring values.

Key Takeaway and What's Next

Key takeaway: Recreating LeNet-5 -- 5x5 kernels, tanh throughout, average pooling (new from-scratch code today, gradient-checked to 4.22e-09 relative error), and three FC layers -- produced a real, correctly-implemented historical architecture with 44,216 parameters. Trained head-to-head against this series' own 2,024-parameter baseline on the identical junction dataset, LeNet5Style reached the same perfect train_acc (1.0000) but a WORSE test_acc (0.7750 vs. 0.8500) -- a direct, measured consequence of 21.8x more parameters chasing only 200 training images. A second, separate finding: LeNet5Style's training is rough early (peak loss 2.81 near epoch 25) but settles to near-total smoothness by the second half, while the baseline's own already-documented oscillation persists to the very end -- a difference traceable to tanh's saturating gradient versus ReLU's non-saturating one. Faithfully recreating a classic architecture is a real, valuable skill; assuming it will outperform a simpler design on every dataset is not the same claim, and today's own numbers show exactly where that assumption breaks.

Suggested watch order, if starting from zero

- 1. "LeNet-5 CNN Architecture Explained | The Network That Started Deep Learning" -- the exact architecture recreated in this lesson, layer by layer.
- 2. "CNN Pooling Explained | Max Pooling, Average Pooling in Convolutional Neural Networks" -- side-by-side coverage of both pooling types this lesson compares.

Day 48 preview

- Transfer Learning -- freezing early layers and fine-tuning a warm-started network, with a real frozen-vs-unfrozen comparison, moving from 'design your own architecture' into 'reuse someone else's already-trained one.'